

UNIVERSIDADE DE MOGI DAS CRUZES
CURSO DE SISTEMAS DE INFORMAÇÃO

LEONARDO DO PRADO ZONZINI

**SISTEMA DE LOGIN SEGURO COM SPRING BOOT, THYMELEAF E
MONGODB ATLAS**

MOGI DAS CRUZES
OUTUBRO DE 2026

LEONARDO DO PRADO ZONZINI

**SISTEMA DE LOGIN SEGURO COM SPRING BOOT, THYMELEAF E
MONGODB ATLAS**

Documentação técnica apresentada à disciplina de Aplicativos Web, do curso de Sistemas de Informação da Universidade de Mogi das Cruzes, referente à Atividade Prática — Parte 1: Login Completo.

Professor: Alessandro Horas

MOGI DAS CRUZES
07 DE OUTUBRO DE 2026

1 INTRODUÇÃO

Este documento descreve o sistema desenvolvido para a Atividade Prática — Parte 1: Login Completo, da disciplina de Aplicativos Web. O trabalho consiste em um sistema de login seguro, construído com Java e Spring Boot, utilizando Thymeleaf para a camada de apresentação e MongoDB Atlas como banco de dados. O sistema contempla cadastro, autenticação e autorização de usuários, com controle de acesso baseado em perfis. Mais do que atender aos requisitos da atividade, o projeto foi estruturado de forma modular e genérica, de modo que sua lógica de autenticação e autorização possa ser reaproveitada em outras aplicações, inclusive no Projeto Final de Curso do autor, sem a necessidade de alterações estruturais relevantes.

Nas seções seguintes são apresentadas a estrutura do sistema, a forma como ele se integra ao MongoDB Atlas e as principais decisões de design adotadas durante o desenvolvimento.

2 ESTRUTURA DO SISTEMA

O projeto segue uma organização em camadas, comum a aplicações Spring Boot, separando claramente responsabilidades de domínio, persistência, regras de negócio, controle de requisições e apresentação. O pacote principal, `com.loginseguro.app`, é dividido nos seguintes subpacotes: `model`, com as classes de domínio `Usuario` e o enum `Papel`; `repository`, com a interface responsável pelo acesso ao MongoDB; `service`, com as regras de cadastro de usuários e a integração com o mecanismo de autenticação do Spring Security; `controller`, com as rotas de autenticação, de registro e dos painéis de cada perfil; e `config`, com a configuração de segurança, de sessão e a criação do usuário administrador inicial.

A autenticação e a autorização são geridas pelo Spring Security. As senhas nunca são armazenadas em texto plano: no cadastro, cada senha é transformada em um hash por meio do algoritmo BCrypt antes de ser persistida. O controle de acesso é feito por perfil de usuário, havendo três perfis distintos — ADMIN, MODERADOR e USUARIO —, cada um com um painel próprio e com regras de acesso às rotas definidas na configuração de segurança. Usuários do perfil ADMIN têm acesso adicional a uma tela de administração, onde é possível alterar o perfil de outros usuários e ativar ou desativar contas.

A camada de apresentação utiliza Thymeleaf com um layout único, montado a partir de fragmentos reutilizáveis — cabeçalho, rodapé e estrutura de página —, de modo que cada tela do sistema herde a mesma composição sem repetir marcação HTML. Essa separação mantém a lógica de negócio nos controllers e serviços, deixando para os templates apenas a apresentação dos dados.

3 INTEGRAÇÃO COM O MONGODB ATLAS

A persistência dos dados é feita em um cluster MongoDB Atlas, acessado por meio do módulo Spring Data MongoDB. Os usuários são armazenados na coleção `usuarios`, com um índice único sobre o campo de e-mail, o que impede o cadastro de contas duplicadas diretamente no nível do banco de dados, além da verificação já feita na camada de serviço.

Além dos usuários, o MongoDB Atlas também armazena as sessões HTTP da aplicação, por meio do módulo Spring Session Data MongoDB. Em vez de manter as sessões apenas em memória, elas são persistidas na coleção `sessoes`, o que evita a perda de sessão em caso de reinício da aplicação e permite que o sistema seja executado em mais de uma instância no futuro, sem que o usuário precise autenticar-se novamente a cada requisição redirecionada para uma instância diferente.

A string de conexão com o cluster não é fixada no código-fonte: ela é lida de uma variável de ambiente (`MONGODB_URI`), configurada separadamente em cada ambiente de execução. Essa abordagem evita a exposição de credenciais do banco de dados no repositório do projeto.

4 DECISÕES DE DESIGN

A primeira decisão relevante foi adotar três perfis de usuário, em vez de um modelo binário de administrador e usuário comum. Essa escolha atende ao requisito mínimo da atividade e também aproxima o sistema de cenários reais, em que nem todo acesso privilegiado equivale a administração completa — o perfil `MODERADOR` representa esse nível intermediário de permissão.

A segunda decisão foi externalizar toda a configuração sensível — conexão com o banco de dados e credenciais do administrador inicial — em variáveis de ambiente, em vez de mantê-la em arquivos de configuração versionados. Um usuário administrador é criado automaticamente na primeira execução da aplicação, a partir dessas variáveis, o que evita a necessidade de inserir manualmente o primeiro usuário privilegiado diretamente no banco de dados.

A terceira decisão foi preparar a camada visual para receber temas diferentes sem alterar a lógica da aplicação. A aparência do sistema está concentrada em um único arquivo de estilos, organizado com variáveis CSS para cores e espaçamentos. Trocar o tema visual do sistema significa substituir esse arquivo — ou apenas os valores das variáveis —, sem modificar nenhum template Thymeleaf ou controller. Essa separação foi pensada especificamente para permitir que o mesmo núcleo de autenticação e autorização seja reaproveitado em outro projeto, adaptando-se apenas a identidade visual ao novo contexto.

5 CONSIDERAÇÕES FINAIS

O sistema desenvolvido atende aos requisitos definidos na atividade: cadastro, login e logout com senha criptografada, controle de acesso por perfil com no mínimo três níveis, uso do Spring Security para autenticação e autorização, persistência de usuários e sessões no MongoDB Atlas, e separação entre a camada visual e a lógica de negócio. A organização modular do código e a configuração baseada em variáveis de ambiente foram escolhidas para que esse mesmo sistema possa, no futuro, ser adaptado a outro domínio de aplicação sem exigir mudanças na lógica central de autenticação e autorização.